VALLEY CROFT

FARM MANAGEMENT SYSTEM

Technical Architecture & Developer Guide

Version 2.0 | March 2026

Prepared by Chynae Digital Solutions

Stack: React.js · Node.js + Express · MongoDB Atlas · Amazon S3

Deployed: Vercel (frontend) · Render (backend) · valleycroft.com

1. Project Overview

Valley Croft Farm Management System is a full-stack web application serving two distinct audiences: the general public booking accommodation and events, and internal staff managing the day-to-day operations of the farm. The system is split into two completely separate repositories deployed independently.

Two separate repos: valleycroft-frontend and valleycroft-backend. Never mix them.

1.1 System Audiences

Audience	Description
Public / Guests	Anyone visiting valleycroft.com — can view rooms, make bookings, track booking status without logging in.
Internal Staff	Admin, Finance, CEO, and Employees — access role-specific dashboards via JWT-authenticated login.

1.2 Internal Roles

Role	Responsibilities
admin	Manages bookings, staff, tasks, inventory, room listings, guest booking confirmations. Full system access.
finance	Manages all financial operations: transactions, salaries, supplier payments, debtors, invoices, refunds, financial statements, audit trail.
ceo	Read-only access to all data: bookings, all financial statements (cashflow, balance sheet, income statement, P&L), staff, debtors, suppliers, reports, audit trail.
employee	Views assigned tasks, submits daily/weekly work logs with photo uploads, views own logs and payslips.

1.3 Repository Structure

Frontend — valleycroft-frontend/

```
valleycroft-frontend/
├── public/
├── src/
│   ├── api/              ← all axios API call functions
│   ├── components/       ← reusable UI components
│   ├── pages/
│   │   ├── public/       ← landing, rooms, booking, tracking
│   │   ├── admin/
│   │   ├── ceo/
│   │   ├── finance/
│   │   └── employee/
```

```
│   ├── context/          ← AuthContext
│   ├── hooks/            ← custom React hooks
│   ├── utils/            ← helpers
│   ├── App.jsx
│   └── main.jsx
├── .env
└── package.json
```

Backend — valleycroft-backend/

```
valleycroft-backend/
├── src/
│   ├── config/           ← db.js (MongoDB connection)
│   ├── controllers/      ← one file per module
│   ├── models/           ← Mongoose schemas
│   ├── routes/           ← Express route files
│   ├── middleware/       ← auth.js, upload.js (S3)
│   └── utils/            ← audit.js, helpers
├── server.js             ← entry point
├── .env
└── package.json
```

1.4 Environment Variables

NEVER commit .env files to GitHub. Add .env to .gitignore before your first commit.

Backend .env

```
PORT=5000
MONGO_URI=mongodb+srv://<user>:<pass>@cluster.mongodb.net/valleycroft
JWT_SECRET=your_very_long_random_secret_key_here
JWT_EXPIRES_IN=7d
AWS_ACCESS_KEY_ID=your_aws_key
AWS_SECRET_ACCESS_KEY=your_aws_secret
AWS_REGION=af-south-1
AWS_S3_BUCKET=valleycroft-uploads
FRONTEND_URL=https://valleycroft.com
```

Frontend .env

```
VITE_API_URL=https://your-backend.onrender.com/api
```

2. Public Landing Page

The public landing page is accessible to anyone without authentication. Guests can browse rooms, make bookings, and track their booking status using a unique tracking code.

2.1 Public Pages

/ (Home)	Hero section, property highlights, featured rooms, call-to-action to book
/rooms	All available rooms with photos (from S3), price per night, amenities, and a Book Now button
/rooms/:id	Single room detail page with full description, photo gallery, date picker, and booking form
/track-booking	Guest enters their email address and tracking code to view current booking status

2.2 Guest Booking Flow

1. Guest selects room and dates on /rooms/:id
2. Frontend sends POST /api/guest-bookings — returns a unique trackingCode
3. Confirmation page displays the trackingCode — instruct guest to save it
4. Admin sees the pending booking in their dashboard and confirms or cancels
5. Guest can check booking status anytime at /track-booking using email + trackingCode

Room images load directly from S3 URLs — the backend is never involved in image delivery. Always use loading='lazy' on room photos.

2.3 Public API Routes — /api/rooms & /api/guest-bookings

Method	Endpoint	Access	Description
GET	/api/rooms	Public	List all available rooms
GET	/api/rooms/:id	Public	Single room detail
POST	/api/guest-bookings	Public	Submit a guest booking request
GET	/api/guest-bookings/track	Public	Track booking by email + trackingCode
POST	/api/rooms	Admin	Add a new room
PUT	/api/rooms/:id	Admin	Update room details or availability
DELETE	/api/rooms/:id	Admin	Remove a room
GET	/api/guest-bookings	Admin, CEO	View all guest booking requests
PUT	/api/guest-bookings/:id	Admin	Confirm, cancel, or update status

3. MongoDB Models (Mongoose Schemas)

Define all schemas first before building any routes. Use these field names consistently across the entire project. All models get timestamps: true.

3.1 User Model

Field	Type	Notes
Name	String, required	Full name
Email	String, required, unique	Login email
Password	String, required	bcrypt hashed — never store plain text
Role	enum: ceo|admin|finance|employee	Controls dashboard access
Phone	String	Optional
idNumber	String	Optional national ID
dateJoined	Date, default: now	
dateLeft	Date	null = still employed
isActive	Boolean, default: true	

3.2 Room Model (new)

Field	Type	Notes
Name	String, required	e.g. 'Garden Suite', 'Event Hall'
Description	String	Full room description for public page
Type	enum: bnb|event-space	
Capacity	Number	Max guests
pricePerNight	Number	
Amenities	[String]	e.g. ['WiFi','Pool','Braai']
Images	[String]	S3 URLs — loaded directly by frontend
isAvailable	Boolean, default: true	Toggle to hide from public page
order	Number	Display order on landing page

3.3 GuestBooking Model (new — public bookings)

Field	Type	Notes
guestName	String, required	
guestEmail	String, required	Used for booking lookup
guestPhone	String	

roomId	ObjectId, ref: Room	Which room was booked
checkIn	Date	
checkOut	Date	
totalAmount	Number	Calculated on submission
deposit	Number	
status	enum: pending|confirmed|cancelled	Admin updates this
trackingCode	String, unique	Random code given to guest for lookup
source	String, default: website	
notes	String	Admin notes

3.4 Booking Model (internal staff bookings)

Field	Type	Notes
guestName	String, required	
guestEmail	String	
guestPhone	String	
type	enum: bnb|event	
checkIn	Date	
checkOut	Date	
eventDate	Date	For event bookings
amount	Number	
deposit	Number	
status	enum: pending|confirmed|checked-in|checked-out|cancelled	
notes	String	
createdBy	ObjectId, ref: User	

3.5 Transaction Model

Field	Type	Notes
type	enum: income|expense	
category	String	booking|salary|supplies|utilities|refund|supplier
description	String	
amount	Number, required	
date	Date, default: now	
reference	String	
booking	ObjectId, ref: Booking	Optional link to booking

createdBy	ObjectId, ref: User	

3.6 Salary & WorkLog Models

Salary

Field	Type	Notes
employee	ObjectId, ref: User, required	
amount	Number	
month	String	e.g. '2026-03'
paidOn	Date	
notes	String	

WorkLog

Field	Type	Notes
employee	ObjectId, ref: User	
date	Date, default: now	
period	enum: daily|weekly	
tasksAssigned	[String]	Set by admin
workDone	String, required	Written by employee
photos	[String]	S3 URLs

3.7 Stock & Equipment Models

Stock

Field	Type	Notes
name	String, required	
category	String	toiletries|cleaning|kitchen
quantity	Number, default: 0	
unit	String	units|litres|kg
reorderLevel	Number	Alert when stock drops below this
lastRestocked	Date	

Equipment

Field	Type	Notes
name	String, required	
category	String	appliance|furniture|machinery

serialNumber	String	
condition	enum: good|fair|needs repair|out of service	
purchaseDate	Date	
lastServiced	Date	
notes	String	

3.8 Finance Models (new)

Debtor

Field	Type	Notes
name	String, required	Guest or company that owes money
contactEmail	String	
contactPhone	String	
description	String	What the debt is for
amountOwed	Number	Original amount
amountPaid	Number, default: 0	Payments made to date
balance	Virtual (amountOwed - amountPaid)	Not stored — computed
dueDate	Date	
status	enum: outstanding|partial|paid|written-off	
bookingRef	ObjectId, ref: Booking	Optional
invoiceRef	ObjectId, ref: Invoice	Optional
notes	String	
createdBy	ObjectId, ref: User	

Supplier

Field	Type	Notes
name	String, required	
contactEmail	String	
contactPhone	String	
category	String	cleaning|food|maintenance|other
bankDetails	Object	{ accountName, bank, accountNumber }
isActive	Boolean, true	
notes	String	
createdBy	ObjectId, ref: User	

SupplierPayment

Field	Type	Notes
supplier	ObjectId, ref: Supplier, required	
amount	Number	
date	Date	
description	String	
invoiceNumber	String	
paymentMethod	enum: cash|EFT|card	
attachmentUrl	String	S3 URL of invoice/receipt scan
createdBy	ObjectId, ref: User	

Invoice

Field	Type	Notes
type	enum: guest|supplier	
relatedTo	ObjectId	Ref to Booking or Supplier
invoiceNumber	String, auto-generated	e.g. INV-2026-0001
issueDate	Date	
dueDate	Date	
lineItems	[{ description, qty, unitPrice, total }]	
subtotal	Number	
Tax	Number	
total	Number	
status	enum: draft|sent|paid|void	
notes	String	
createdBy	ObjectId, ref: User	

Refund

Field	Type	Notes
guestName	String	
guestEmail	String	
bookingRef	ObjectId	Ref to Booking or GuestBooking
amount	Number	
reason	String	Why the refund is being issued
status	enum: pending|approved|processed|rejected	
processedBy	ObjectId, ref: User	Finance user who actioned it

processedOn	Date	
notes	String	
createdBy	ObjectId, ref: User	

3.9 AuditLog Model (new — all dashboards)

Field	Type	Notes
userId	ObjectId, ref: User, required	Who performed the action
Role	String	Their role at time of action
action	enum: create|update|delete|login|logout|export	
entity	String	e.g. 'Booking', 'Transaction', 'Refund'
entityId	ObjectId	Which record was affected
before	Mixed	Snapshot of data before change
after	Mixed	Snapshot of data after change
Ip	String	Request IP address
userAgent	String	Browser/device info
timestamp	Date, default: now	Index on this field

AuditLog documents must NEVER be deleted or updated. The collection is append-only. Remove all DELETE and PUT routes from /api/audit.

Indexes — add to each model file

```
BookingSchema.index({ checkIn: 1, checkOut: 1 });
BookingSchema.index({ status: 1 });
TransactionSchema.index({ date: -1 });
TransactionSchema.index({ type: 1, date: -1 });
WorkLogSchema.index({ employee: 1, date: -1 });
AuditLogSchema.index({ userId: 1, timestamp: -1 });
AuditLogSchema.index({ entity: 1, timestamp: -1 });
DebtorSchema.index({ status: 1 });
GuestBookingSchema.index({ trackingCode: 1 });
GuestBookingSchema.index({ guestEmail: 1 });
```

4. Backend Setup (Node.js + Express)

4.1 Install Packages

```
npm install express mongoose dotenv cors bcryptjs jsonwebtoken compression helmet
npm install multer @aws-sdk/client-s3 multer-s3
npm install --save-dev nodemon
```

4.2 server.js Entry Point

```
const express = require('express');
const mongoose = require('mongoose');
const cors = require('cors');
const helmet = require('helmet');
const compression = require('compression');
require('dotenv').config();

const app = express();
app.use(helmet());
app.use(compression());
app.use(cors({ origin: process.env.FRONTEND_URL }));
app.use(express.json());

// Public routes (no auth)
app.use('/api/rooms',          require('./src/routes/roomRoutes'));
app.use('/api/guest-bookings', require('./src/routes/guestBookingRoutes'));

// Protected routes
app.use('/api/auth',           require('./src/routes/authRoutes'));
app.use('/api/bookings',       require('./src/routes/bookingRoutes'));
app.use('/api/finance',        require('./src/routes/financeRoutes'));
app.use('/api/staff',          require('./src/routes/staffRoutes'));
app.use('/api/inventory',      require('./src/routes/inventoryRoutes'));
app.use('/api/reports',        require('./src/routes/reportRoutes'));
app.use('/api/debtors',        require('./src/routes/debtorRoutes'));
app.use('/api/suppliers',      require('./src/routes/supplierRoutes'));
app.use('/api/invoices',       require('./src/routes/invoiceRoutes'));
app.use('/api/refunds',        require('./src/routes/refundRoutes'));
app.use('/api/audit',          require('./src/routes/auditRoutes'));

// Health check — critical for Render keepalive
app.get('/api/health', (req, res) => res.json({ status: 'ok' }));
```

Add uptime monitor at uptimerobot.com (free) pinging /api/health every 10 minutes. Staff must never experience a 20–30 second cold start in the morning.

4.3 Authentication Middleware

Build this first — every other module depends on it.

```
// src/middleware/auth.js
const jwt = require('jsonwebtoken');

const protect = (req, res, next) => {
  const token = req.headers.authorization?.split(' ')[1];
  if (!token) return res.status(401).json({ message: 'Not authorised' });
  try {
    req.user = jwt.verify(token, process.env.JWT_SECRET);
    next();
  } catch { res.status(401).json({ message: 'Invalid token' }); }
};

const authorize = (...roles) => (req, res, next) => {
  if (!roles.includes(req.user.role))
    return res.status(403).json({ message: 'Access denied' });
  next();
};

module.exports = { protect, authorize };
```

4.4 Audit Utility

Wire this into EVERY controller that creates, updates, or deletes data. Also call on login, logout, and PDF exports.

```
// src/utils/audit.js
const AuditLog = require('../models/AuditLog');

const logAudit = async ({ userId, role, action, entity, entityId, before, after, req
}) => {
  await AuditLog.create({
    userId, role, action, entity, entityId,
    before, after,
    ip: req?.ip,
    userAgent: req?.headers['user-agent'],
  });
};

module.exports = logAudit;
```

4.5 Performance Rules

- Add .lean() to EVERY GET query — returns plain JS objects, up to 40% faster
- Use .select() on every query — never return password field, only return fields the frontend needs
- All list endpoints must support pagination: ?page=1&limit=20 (default limit 20)
- Use MongoDB aggregation pipelines for ALL financial reports — never load data into Node and compute in JS
- Use compression middleware before all routes — reduces response size 60–70%
- Standard response shape everywhere: { success: true/false, data: ..., message: '...', meta: { page, limit, total } }

5. API Routes — Module by Module

Convention: All routes prefixed with /api/. Apply protect on every non-public route. Apply authorize to restrict by role.

5.1 Auth Routes — /api/auth

Method	Endpoint	Access	Description
POST	/login	Public	Login — returns JWT token
POST	/register	Admin only	Create new user account
GET	/me	All logged in	Get current logged-in user
PUT	/change-password	All logged in	Change own password

5.2 Booking Routes — /api/bookings (internal)

Method	Endpoint	Access	Description
GET	/	Admin, CEO	Get all internal bookings
GET	/availability	Admin, CEO	Check BnB / event availability
GET	/:id	Admin, CEO	Get a single booking
POST	/	Admin	Create new internal booking
PUT	/:id	Admin	Update booking or status
DELETE	/:id	Admin	Cancel or delete booking

5.3 Finance Routes — /api/finance

Method	Endpoint	Access	Description
GET	/transactions	Finance, Admin, CEO	All income & expense records
POST	/transactions	Finance, Admin	Add income or expense entry
PUT	/transactions/:id	Finance, Admin	Edit a transaction
DELETE	/transactions/:id	Finance, Admin	Delete a transaction
GET	/cashflow	Finance, Admin, CEO	Cash flow by date range
GET	/income-statement	Finance, CEO	Income statement by period
GET	/balance-sheet	Finance, CEO	Balance sheet snapshot
GET	/pl	Finance, CEO	Profit & Loss statement

GET	/salary	Finance, Admin, CEO	All salary payment records
POST	/salary	Finance, Admin	Record a salary payment
GET	/salary/employee/:id	Finance, Admin	Salary history for one employee

5.4 Debtors Routes — /api/debtors

Method	Endpoint	Access	Description
GET	/	Finance, Admin, CEO	All debtor records
POST	/	Finance, Admin	Create a debtor record
PUT	/:id	Finance, Admin	Update payment or status
DELETE	/:id	Finance, Admin	Write off or remove debtor

5.5 Supplier Routes — /api/suppliers

Method	Endpoint	Access	Description
GET	/	Finance, Admin, CEO	All suppliers
POST	/	Finance, Admin	Add a supplier
PUT	/:id	Finance, Admin	Update supplier details
GET	/:id/payments	Finance, Admin, CEO	Payment history for supplier
POST	/payments	Finance, Admin	Record a supplier payment

5.6 Invoice Routes — /api/invoices

Method	Endpoint	Access	Description
GET	/	Finance, Admin, CEO	All invoices
POST	/	Finance, Admin	Create an invoice
PUT	/:id	Finance, Admin	Update invoice status
GET	/:id/pdf	Finance, Admin, CEO	Download invoice as PDF

5.7 Refund Routes — /api/refunds

Method	Endpoint	Access	Description
GET	/	Finance, Admin, CEO	All refund records
POST	/	Finance, Admin	Create a refund request

PUT	/:id	Finance, Admin	Approve, process, or reject refund

5.8 Staff Routes — /api/staff

Method	Endpoint	Access	Description
GET	/employees	Admin, CEO	List all employees
PUT	/employees/:id	Admin	Update profile / set date left
POST	/tasks	Admin	Assign task to employee
GET	/tasks/:employeeId	Admin, CEO, Employee	Get tasks for employee
GET	/worklogs	Admin, CEO	View all work logs
GET	/worklogs/me	Employee only	Employee views own logs
POST	/worklogs	Employee only	Submit work log + photos (S3)

Employee can only fetch their own tasks — enforce this with req.user._id in the controller, never trust the URL param.

5.9 Inventory Routes — /api/inventory

Method	Endpoint	Access	Description
GET	/stock	Admin, CEO	All consumable stock items
POST	/stock	Admin	Add new stock item
PUT	/stock/:id	Admin	Update quantity or restock
DELETE	/stock/:id	Admin	Remove stock item
GET	/equipment	Admin, CEO	Full equipment register
POST	/equipment	Admin	Add equipment
PUT	/equipment/:id	Admin	Update condition or service date

5.10 Reports Routes — /api/reports

Method	Endpoint	Access	Description
GET	/weekly	Admin, Finance, CEO	Weekly summary report
GET	/monthly	Admin, Finance, CEO	Monthly detailed report
GET	/quarterly	Admin, Finance, CEO	Quarterly business report
GET	/annual	Admin, Finance, CEO	Full year report
GET	/export/:type	Admin, Finance, CEO	Download report as PDF

5.11 Audit Routes — /api/audit

Method	Endpoint	Access	Description
GET	/	Admin, CEO, Finance	Query audit log by user, entity, or date range
GET	/entity/:name	Admin, CEO, Finance	All actions on a specific entity type
GET	/user/:id	Admin, CEO	All actions by a specific user

No POST, PUT, or DELETE routes on /api/audit — ever. The audit log is read-only and append-only.

6. Frontend Setup (React + Vite)

6.1 Initialize & Install

```
npm create vite@latest valleycroft-frontend -- --template react
cd valleycroft-frontend && npm install
npm install axios react-router-dom@^6.26.0 react-hook-form
npm install @tanstack/react-query date-fns recharts
npm install tailwindcss @tailwindcss/vite
```

Use react-router-dom v6 — v7 has compatibility issues with Vercel deployments. Pin to ^6.26.0 in package.json.

6.2 Routing Structure

```
// Public routes (no auth required)
<Route path='/'            element={<LandingPage />} />
<Route path='/rooms'       element={<RoomsPage />} />
<Route path='/rooms/:id'   element={<RoomDetail />} />
<Route path='/track-booking' element={<TrackBooking />} />

// Protected routes (role-gated)
<Route path='/login' element={<Login />} />
<Route element={<ProtectedRoute roles={['ceo']} />}>
  <Route path='/ceo/*' element={<CeoDashboard />} />
</Route>
<Route element={<ProtectedRoute roles={['admin']} />}>
  <Route path='/admin/*' element={<AdminDashboard />} />
</Route>
<Route element={<ProtectedRoute roles={['finance']} />}>
  <Route path='/finance/*' element={<FinanceDashboard />} />
</Route>
<Route element={<ProtectedRoute roles={['employee']} />}>
  <Route path='/employee/*' element={<EmployeeDashboard />} />
</Route>
```

6.3 Dashboard Pages by Role

Admin Dashboard

/admin/dashboard	Today's bookings, low stock alerts, guest booking requests, staff on duty
/admin/bookings	Internal booking calendar, create/edit/cancel, availability view
/admin/guest-bookings	Guest booking requests from website — confirm or cancel
/admin/rooms	Manage room listings, photos, availability toggle
/admin/staff	Employee list, add employee, assign tasks, view work logs
/admin/salary	Record salary payments per employee
/admin/inventory	Stock levels, reorder alerts, equipment register

/admin/reports	Generate and download reports
/admin/audit	Full audit trail — all users and actions

Finance Dashboard

/finance/dashboard	Income vs expense summary, month-to-date totals
/finance/transactions	Add income/expense, view and filter all entries
/finance/salary	Record salary payments, filter by employee or month
/finance/suppliers	Supplier list, add payment, payment history per supplier
/finance/debtors	Outstanding debtors, mark partial/paid, aging summary
/finance/invoices	Create invoice, change status, download PDF
/finance/refunds	Pending refunds queue, approve/reject/process
/finance/cashflow	Cash flow statement with date range filter
/finance/income-statement	Income statement broken down by revenue stream
/finance/balance-sheet	Balance sheet view
/finance/pl	Profit & Loss statement
/finance/audit	Audit trail filtered to finance-relevant actions

CEO Dashboard (all read-only)

/ceo/dashboard	Full business overview: revenue, occupancy, headcount, alerts
/ceo/bookings	Read-only bookings view
/ceo/finance	Financial summary with all statements
/ceo/cashflow	Cash flow statement with date range
/ceo/income-statement	Income statement by period
/ceo/balance-sheet	Balance sheet snapshot
/ceo/pl	Profit & Loss statement
/ceo/debtors	Debtors summary with aging
/ceo/suppliers	Supplier payment summary
/ceo/staff	Read-only staff profiles and work logs
/ceo/reports	Access all reports, download PDFs
/ceo/audit	Full audit trail, read-only

Employee Dashboard

/employee/dashboard	Tasks assigned to me, pending submissions
/employee/log-work	Form to submit daily/weekly work log and photo upload
/employee/my-logs	History of my own past work log submissions
/employee/payslips	View own salary payment history

6.4 Axios Instance

```
// src/api/axiosInstance.js
import axios from 'axios';

const api = axios.create({ baseURL: import.meta.env.VITE_API_URL });

api.interceptors.request.use((config) => {
  const token = localStorage.getItem('token');
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});

export default api;
```

6.5 React Query Setup

```
// src/main.jsx
import { QueryClient, QueryClientProvider } from '@tanstack/react-query';

const queryClient = new QueryClient({
  defaultOptions: {
    queries: { staleTime: 1000 * 60 * 5, retry: 1 },
  },
});

// Wrap <App /> with <QueryClientProvider client={queryClient}>
```

React Query is the single biggest speed win on the frontend. A user who navigates away and returns gets instant data from cache — zero loading spinner, zero API call for 5 minutes.

7. Performance — Building a 2x Faster System

Apply these techniques as you build each module — not as an afterthought.

7.1 Frontend Performance

Code splitting — lazy load each dashboard

```
const AdminDashboard    = lazy(() => import('./pages/admin/AdminDashboard'));
const FinanceDashboard  = lazy(() => import('./pages/finance/FinanceDashboard'));
const CeoDashboard      = lazy(() => import('./pages/ceo/CeoDashboard'));
const EmployeeDashboard = lazy(() => import('./pages/employee/EmployeeDashboard'));
// Wrap routes in <Suspense fallback={<LoadingSpinner />}>
```

Optimistic UI updates

```
const mutation = useMutation({
  mutationFn: (newBooking) => api.post('/bookings', newBooking),
  onMutate: async (newBooking) => {
    queryClient.setQueryData(['bookings'], (old) => [...old, newBooking]);
  },
  onError: () => { queryClient.invalidateQueries(['bookings']); },
});
```

Pagination — never load all records at once

```
const { data } = useQuery({
  queryKey: ['bookings', page],
  queryFn: () => api.get(`/bookings?page=${page}&limit=20`),
  keepPreviousData: true, // no flicker between pages
});
```

7.2 Backend Performance

Lean queries — 40% faster reads

```
// Without lean — slow
const bookings = await Booking.find({ status: 'confirmed' });

// With lean — fast
const bookings = await Booking.find({ status: 'confirmed' }).lean();
```

Field projection — smaller payloads

```
const employees = await User
  .find({ role: 'employee' })
  .select('name email phone role dateJoined isActive')
  .lean();
```

Aggregation for financial reports

```
const incomeStatement = await Transaction.aggregate([
  { $match: { date: { $gte: startDate, $lte: endDate }, type: 'income' }},
  { $group: { _id: '$category', total: { $sum: '$amount' }, count: { $sum: 1 }}},
  { $sort: { total: -1 } }
]);
```

7.3 Infrastructure Performance

S3 images — direct delivery, not through backend

```
// Frontend loads S3 URL directly — backend handles zero image traffic
<img src={room.images[0]} alt='Room photo' loading='lazy' />
```

This is critical for Valley Croft — staff and guests opening the app on mobile in South Africa should never wait for a cold start or slow image load.

8. Recommended Build Order

Follow this exact order. Each step builds on the previous. Do not skip ahead — authentication must be working before any other module.

#	What to Build	Details	Module
1	Backend Init + MongoDB	Create project, install packages, connect to MongoDB Atlas, test connection logs	Foundation
2	User Model + Auth API	User schema, /login, /register (admin only), JWT generation, protect & authorize middleware	Auth
3	Room Model + /api/rooms	Room schema, public GET all/one, admin CRUD, S3 image URLs	Rooms
4	GuestBooking + Public Pages	GuestBooking model, POST booking, tracking endpoint, landing page, /rooms, /rooms/:id, /track-booking	Public Site
5	React App + Login	Vite app, packages, login form, store token, ProtectedRoute, role redirects	Frontend Base
6	Admin: Guest Bookings + Rooms	Admin view of guest booking requests, confirm/cancel, manage room listings	Admin
7	Internal Bookings Module	Booking model, all CRUD, availability logic, Admin booking pages, calendar UI	Module 1
8	Finance: Core Transactions	Transaction model, income/expense routes, Finance dashboard pages	Module 2a
9	Finance: Salary + Suppliers	Salary model, SupplierPayment model, record salary, supplier list, payment history	Module 2b
10	Finance: Debtors + Invoices + Refunds	Debtor/Invoice/Refund models, all CRUD routes, invoice PDF generation	Module 2c
11	Finance: Statements	Aggregation pipelines for cashflow, income statement, balance sheet, P&L — CEO and Finance pages	Module 2d
12	Audit Trail	AuditLog model, logAudit utility, wire into ALL existing controllers, /api/audit read-only routes, audit pages per dashboard	Cross-cutting
13	Staff Module	Employee profiles, salary history via Auth, task assignment, WorkLog model, S3 upload middleware, employee pages	Module 3
14	Inventory Module	Stock model, Equipment model, all CRUD routes, reorder alert logic, Admin inventory pages	Module 4
15	Reports Module	MongoDB aggregation pipelines for each period, PDF export using pdfkit or puppeteer	Module 5
16	CEO Dashboard	Read-only views wired to all existing API endpoints, consolidated	Module 6

		overview, all financial statements, charts with Recharts	
17	Deploy Everything	Push to GitHub, deploy backend on Render, deploy frontend on Vercel, connect valleycroft.com	Go Live

9. Deployment Guide

9.1 Deploy Backend to Render

- Push valleycroft-backend to a GitHub repository
- Go to render.com → New Web Service → connect your GitHub repo
- Build command: npm install
- Start command: node src/server.js
- Add all .env variables in Render's Environment section
- Copy the Render URL (e.g. https://valleycroft-api.onrender.com) — you'll need it for the frontend

Set up uptimerobot.com (free) to ping /api/health every 10 minutes. Without this, the first request of the morning will take 20–30 seconds.

9.2 Deploy Frontend to Vercel

- Push valleycroft-frontend to a GitHub repository
- Go to vercel.com → New Project → import the repo
- Framework preset: Vite
- Build command: npm run build | Output directory: dist
- Add environment variable: VITE_API_URL = your Render backend URL + /api
- Vercel auto-deploys on every push to the main branch

Ensure Node version is set to 20.x in package.json engines field. Node 24 is not yet supported on Vercel.

9.3 Connect valleycroft.com

- In Vercel: project → Settings → Domains → add valleycroft.com
- In your domain registrar: add the DNS records Vercel provides
- SSL certificate is handled automatically — no manual setup needed
- DNS propagation can take 10 minutes to 48 hours

9.4 MongoDB Atlas Setup

- Create account at mongodb.com/atlas → New Project → Build a Database
- Create a database user with a strong password — save these credentials
- Under Network Access: add 0.0.0.0/0 to allow connections from Render
- Copy the connection string → paste into Render's MONGO_URI environment variable

10. Best Practices & Key Rules

Security

- Always hash passwords with bcrypt before saving — never store plain text
- Set JWT expiry to 7 days (JWT_EXPIRES_IN env var)
- Validate all request body inputs on the backend — never trust frontend data
- Set CORS to only allow your Vercel domain — never wildcard (*)
- Use helmet() in server.js on all routes
- Never return the password field in any API response — use .select('-password')
- Never log tokens, passwords, or S3 keys

Finance & Audit Rules

- Every write operation (create, update, delete) must call logAudit() — no exceptions
- AuditLog collection is append-only — never delete or update audit records
- All financial statement calculations happen in MongoDB aggregation pipelines — never in Node.js
- Invoices should be immutable once marked 'sent' — only status can be changed after that
- Refunds must have a corresponding transaction record created when status is set to 'processed'

Developer Workflow

- Use Postman or Insomnia to test every API route before connecting the frontend
- Build and fully test each module (backend + frontend) before moving to the next
- Use Git branches: one branch per module e.g. feature/bookings — merge to main when done
- Test on mobile throughout development — guests and staff will access this on phones
- Build backend API first → test in Postman → then build the matching frontend pages
- Never build backend and frontend for the same module simultaneously

Performance Checklist

- All GET queries use .lean()
- All GET queries use .select() — only return needed fields
- All list endpoints are paginated: ?page=1&limit=20
- MongoDB indexes added on all frequently queried fields before go-live
- React Query wraps all data fetching with 5-minute staleTime
- All dashboard components have loading, error, and empty states
- Room and work log images load directly from S3 — never through the backend
- All dashboard pages use React.lazy() + Suspense code splitting

Golden rule: Build the backend API first → test it in Postman → then build the matching frontend pages. Never build backend and frontend for the same module simultaneously.